

Java 8 Collectors API in-depth

Collectors is a **public final class** that extends **Object class**.

- Collectors class provides various useful reduction operations, such as accumulating elements into collections, summarizing elements according to various criteria, etc.

Java 8 Collectors Methods

◇ Basic

Method	Description	Example
<code>toList()</code>	Collect elements into List	<code>stream.collect(Collectors.toList())</code>
<code>toSet()</code>	Collect elements into Set	<code>stream.collect(Collectors.toSet())</code>
<code>toCollection()</code>	Custom collection	<code>Collectors.toCollection(ArrayList::new)</code>

◇ Map

Method	Description	Example
<code>toMap()</code>	Convert stream to Map	<code>Collectors.toMap(k,v)</code>
<code>toMap() (merge)</code>	Handle duplicate keys	<code>Collectors.toMap(k,v,(a,b)->a+b)</code>
<code>toConcurrentMap()</code>	Thread-safe map	<code>Collectors.toConcurrentMap(k,v)</code>

◇ Grouping & Partitioning

Method	Description	Example
<code>groupingBy()</code>	Group elements by key	<code>Collectors.groupingBy(String::length)</code>
<code>groupingByConcurrent()</code>	Concurrent grouping	<code>Collectors.groupingByConcurrent(...)</code>
<code>partitioningBy()</code>	Split into two groups	<code>Collectors.partitioningBy(x->x%2==0)</code>

◇ Counting & Statistics

Method	Description	Example
<code>counting()</code>	Count elements	<code>Collectors.counting()</code>
<code>summingInt()</code>	Sum int values	<code>Collectors.summingInt(x->x)</code>
<code>summingLong()</code>	Sum long values	<code>Collectors.summingLong(x->x)</code>
<code>summingDouble()</code>	Sum double values	<code>Collectors.summingDouble(x->x)</code>
<code>averagingInt()</code>	Average int values	<code>Collectors.averagingInt(x->x)</code>

Method	Description	Example
<code>averagingLong()</code>	Average long values	<code>Collectors.averagingLong(x->x)</code>
<code>averagingDouble()</code>	Average double values	<code>Collectors.averagingDouble(x->x)</code>
<code>summarizingInt()</code>	Int summary statistics	<code>Collectors.summarizingInt(x->x)</code>
<code>summarizingLong()</code>	Long summary statistics	<code>Collectors.summarizingLong(x->x)</code>
<code>summarizingDouble()</code>	Double summary statistics	<code>Collectors.summarizingDouble(x->x)</code>

◇ Reduction & Transformation

Method	Description	Example
<code>reducing()</code>	General reduction	<code>Collectors.reducing(0, Integer::sum)</code>
<code>mapping()</code>	Apply mapping before collect	<code>Collectors.mapping(String::length, Collectors.toList())</code>
<code>collectingAndThen()</code>	Modify final result	<code>Collectors.collectingAndThen(toList(), Collections::unmodifiableList)</code>

◇ String Operations

Method	Description	Example
<code>joining()</code>	Join elements	<code>Collectors.joining()</code>
<code>joining(delimiter)</code>	Join with delimiter	<code>Collectors.joining(",")</code>
<code>joining(delimiter,prefix,suffix)</code>	Full control join	<code>Collectors.joining(",", "[", "]")</code>

◇ Min / Max

Method	Description	Example
<code>maxBy()</code>	Find max element	<code>Collectors.maxBy(Comparator.naturalOrder())</code>
<code>minBy()</code>	Find min element	<code>Collectors.minBy(Comparator.naturalOrder())</code>

📌 Note

All methods in the `Collectors` class are static.

👉 So, it is good to use static import.

We will be using the below `Employee` class

```
class Employee {
    private int id;
    private String name;
}
```

```
private int age;
private String region;
private double sal;

public Employee(int id, String name, int age, String region, double sal) {
    this.id = id;
    this.name = name;
    this.age = age;
    this.region = region;
    this.sal = sal;
}

// Standard setters and getters
}
```

Creating an Employee List.

```
List<Employee> employeeList = new ArrayList<>();
employeeList.add(new Employee(100, "Sundar", 47, "North America", 450000));
employeeList.add(new Employee(200, "Pichai", 25, "North America", 50000));
employeeList.add(new Employee(300, "Larry", 30, "Asia", 450000));
employeeList.add(new Employee(400, "Page", 59, "Africa", 450000));
```

Java 8 Stream.collect() Example

- Java 8's most powerful stream method is collect() method. Which is also called a Terminal method. This is part of Stream API.
- It allows us to perform mutable fold operations (repackaging elements to some data structures and applying some additional logic, concatenating them, etc.) on data elements held in a Stream instance.
- The strategy for this operation is provided via Collector interface implementation.

Collectors.toList() Example

- toList() collector can be used for collecting all Stream elements into a List instance.
- Example to collect all employee names into List using toList() method.

```
List<String> namesList = employeeList.stream()
                                    .map(e -> e.getName())
                                    .collect(Collectors.toList());

System.out.println(namesList);
```

****Output:** ● [Sundar, Pichai, Larry, Page]**

- But, there are no guarantees on the type, mutability, serializability, or thread-safety of the List returned.
- If you need more control over what type of List should be returned then should use `toCollection(Supplier)` method.

Collectors.toSet() Example

`toSet()` collector is used for collecting all Stream elements into a Set instance.

Example to collect all the regions into Set.

```
Set<String> regionSet = employeeList.stream()
                                   .map(e -> e.getRegion())
                                   .collect(Collectors.toSet());

System.out.println(regionSet);

**Output: [Asia, Africa, North America]**
```

- But, there are no guarantees on the type, mutability, serializability, or thread-safety of the Set returned.

🔗 Note If you need more control over what type of Set should be returned then should use the `toCollection(Supplier)` method.

Collectors.toUnmodifiableSet() Example

- This collects the elements into an unmodifiable set.
- The set is created using the `toSet()` method can be modified.

```
Set<String> unmodifiableSet = employeeList.stream()
                                       .map(e -> e.getSal())
                                       .collect(Collectors.toUnmodifiableSet());

System.out.println(unmodifiableSet);

**Output:** [45000.0, 50000.0]
```

If we try to modify set then will throw **UnsupportedOperationException**.

`unmodifiableSet.add(10983d);`

```
Exception in thread "main" java.lang.UnsupportedOperationException
```

The returned collector does not allow null values. This will throw `NullPointerException` if it is presented with a null value.

```
employeeList.add(null);
Set<Employee> empSet =
employeeList.stream().collect(Collectors.toUnmodifiableSet());
```

****Output:**** ● The above code will cause `NullPointerException`. The same will be in the `case` of the `toSet()` method

Collectors.toUnmodifiableList() Example

- This is similar to the `toList()` but `toUnmodifiableList` will collect elements into an unmodifiable List.

```
Set<String> unmodifiableList = employeeList.stream()
                                     .map(e -> e.getSal())
                                     .collect(Collectors.toUnmodifiableList());
System.out.println(unmodifiableList );
```

****Output:**** ● [450000.0, 50000.0, 450000.0, 450000.0]

- This list holds duplicates, unlike Set.

✂ If List has null value then it will throw `java.lang.NullPointerException` like `toUnmodifiableSet`.

Collectors.toCollection() Example

- As you probably already noticed, when using `toSet()` and `toList()` collectors, you can't make any assumptions of their implementations.
- If you want to use a custom implementation or `LinkedList` or `TreeSet`, you will need to use the `toCollection` collector with a provided collection of your choice.
- Example to collect the names into `LinkedList` as opposed to default List implementation.

```
Set<String> namesLinkedList = employeeList.stream()
                                     .map(e -> e.getName())
                                     .collect(Collectors.toCollection(LinkedList::new));
System.out.println(namesLinkedList );
```

****Output:**** ● [Sundar, Pichai, Larry, Page]

Another example to collect regions into TreeSet.

```
Set<String> regionTreeSet = employeeList.stream()
                                     .map(e -> e.getRegion())
```

```

                .collect(Collectors.toCollection
                (TreeSet::new));

System.out.println(regionTreeSet);
**Output:** [Africa, Asia, North America]

```

 **Note:** This method does not work with immutable objects. In such type of cases, We must write custom Collector implementation or Use `collectingAndThen()`.

Collectors.toMap() Example

- **toMap() Syntax:**

```

public static <T,K,U> Collector<T,?,Map<K,U>> toMap(Function<? super T,? extends
K> keyMapper, Function<? super T,? extends U> valueMapper)

```

- Using `toMap()` method, A stream can be converted into a Map. But, this method needs two parameters.
- **keyMapper**
- **valueMapper**
- These two are implementations of Function Functional Interface.
- Functional Interface Function has a functional method `R apply(T t)` that accepts one argument and produces a result.
- `keyMapper` will be used for extracting a Map key from a Stream element, and `valueMapper` will be used for extracting a value associated with a given key.
- Now, We will create a map from a stream such that the map key is emp id and value is corresponding employee object.
- `keyMapper = (e) -> e.getId()` `e` refers to the Employee object and getting its id by calling `getId()` method.
- `valueMapper = Function.identity()` This method returns a function that always returns its input argument.
- `Function.identity()` method does take one object as an argument and returns the same object with no change.

```

Map<Integer, Employee> empMap = employeeList.stream()
                .collect(Collectors.toMap((e) -> e.getId(),
                Function.identity()));

System.out.println(empMap);

**Output:** {400=Employee [id=400, name=Page, age=59, region=Africa, sal=450000.0],

```

```
100=Employee [id=100, name=Sundar, age=47, region=North America, sal=450000.0],
200=Employee [id=200, name=Pichai, age=25, region=North America, sal=50000.0],
300=Employee [id=300, name=Larry, age=30, region=Asia, sal=450000.0]}
```

- What happens if employeeList has duplicate employees with the same employee id.
- Now adding a duplicate emp object with the same emp id but the different name "Larry Page".

```
employeeList.add(new Employee(400, "Larry Page", 59, "Africa", 450000));

Map<Integer, Employee> empDupMap = employeeList.stream()
    .collect(Collectors.toMap((e) -> e.getId(),
        Function.identity()));
```

Will throw the Runtime Exception as follow.

```
Exception in thread "main" java.lang.IllegalStateException: Duplicate key 400
(attempted merging values Employee [id=400, name=Page, age=59, region=Africa,
sal=450000.0] and Employee [id=400, name=Page, age=59, region=Africa,
sal=450000.0])
at
java.base/java.util.stream.Collectors.duplicateKeyException(Collectors.java:133)
at
java.base/java.util.stream.Collectors.lambda$uniqKeysMapAccumulator$1(Collectors.j
ava:180)
```

- toMap() function takes 3rd argument as BinaryOperator Functional Interface which has a functional method R apply(T t, U u).
- This functional method takes two arguments. In our case, the first argument takes the original employee, Second argument takes the duplicate employee and returns the employee object.

```
Map<Integer, Employee> empDupMap = employeeList.stream()
    .collect(Collectors.toMap((e) -> e.getId(),
        Function.identity(),
        (emp, sameEmp) -> sameEmp));

System.out.println(empDupMap);
```